

Web Systems Development

Week 5 React



September 15, 2026

Agenda

- Timeline
 - React Review
 - React Part II

Upcoming Agenda

5	Tue 9/22 · Fri 9/25	React — components and the render model	In-class — Tue 9/22 HW 3 — Fri 9/25 Project pitch — Fri 9/25
6	Tue 9/29 · Fri 10/2	React — state, composition, component testing	In-class — Tue 9/29 HW 4 — Fri 10/2
7	Tue 10/6 · Fri 10/9	Laravel — what a framework buys you	In-class — Tue 10/6 HW 5 — Fri 10/9
8	Tue 10/13	Quiz 1 — handwritten, closed-device	Quiz 1 — Tue 10/13
—	<i>Fri 10/16</i>	<i>No class — follows a Monday schedule</i>	—

ITWS 2110 · WEEK 6 · SESSIONS 10 AND 11

React Part I Review

WHAT A COMPONENT IS

A function that returns JSX

```
function Product({ name, expired }) {  
  return (  
    <li className="flex justify-between">  
      <span>{name}</span>  
      <span>{expired ? 'Expired' : 'Fresh'}</span>  
    </li>  
  );  
}  
  
<ul>  
  <Product name="Apples" expired={false} />  
  <Product name="Rice"    expired={true} />  
</ul>
```

- It's a JavaScript function. Its name starts with a capital: `<Product />` is yours; `<product />` is treated as an HTML tag.
- It takes one argument, props, and returns JSX — a description of what should be on screen.
- You don't call it. You write `<Product />` and React calls it: once per instance, and again whenever its data changes.
- One definition, any number of instances, each with its own props.

Props in, JSX out. A component describes the screen for the data it's given — it never edits the page itself.

Last week's drills 4 (props) and 5 (lists). Your HW4 PantryCard is exactly this shape.

Every part of a component you wrote last week

```
import { useState } from 'react';           // 1

export function Counter({ label }) {        // 2 3
  const [count, setCount] = useState(0);    // 4

  function handleClick() {                  // 5
    setCount(c => c + 1);
  }

  return (                                  // 6
    <button className="rounded border px-4"
      onClick={handleClick}>                // 7
      {label}: {count}
    </button>
  );
}
```

- 1 import**
hooks come from React; your components come from your own files
- 2 export function Counter**
PascalCase, and exported so another file can use it
- 3 { label }**
props, destructured. Read-only here — the parent decides them
- 4 useState(0)**
memory that survives a re-render. Top level, every render
- 5 handleClick**
a handler: it calls the setter, never assigns to count
- 6 return (...)**
JSX. className, not class; {braces} for any JS expression
- 7 onClick={handleClick}**
pass the function; don't call it with ()

Seven parts, and you've used every one. Today adds no new syntax — only decisions about where 4 belongs.

Who owns this value?

Props

```
<Product name="Rice" />
```

- Passed in by the parent, like function arguments
- Read-only inside the component
- Change only when the parent renders again with new ones
- Flow down: parent → child

State

```
const [count, setCount] = useState(0);
```

- Owned by the component that calls useState
- Changed only through its setter
- Survives re-renders; every instance has its own
- Changes go up as calls: the child calls onAdd, the parent sets

One component's state is often another component's props. In last week's drill 9, the parent's count is each child's count prop.

In React, **props** (short for "properties") are read-only arguments passed into components to allow them to communicate and share data.

What is a hook?

A hook is a function whose name starts with use that lets a component use React features, such as state.

What is children?

children is a special prop that contains whatever you place between a component's opening and closing tags.

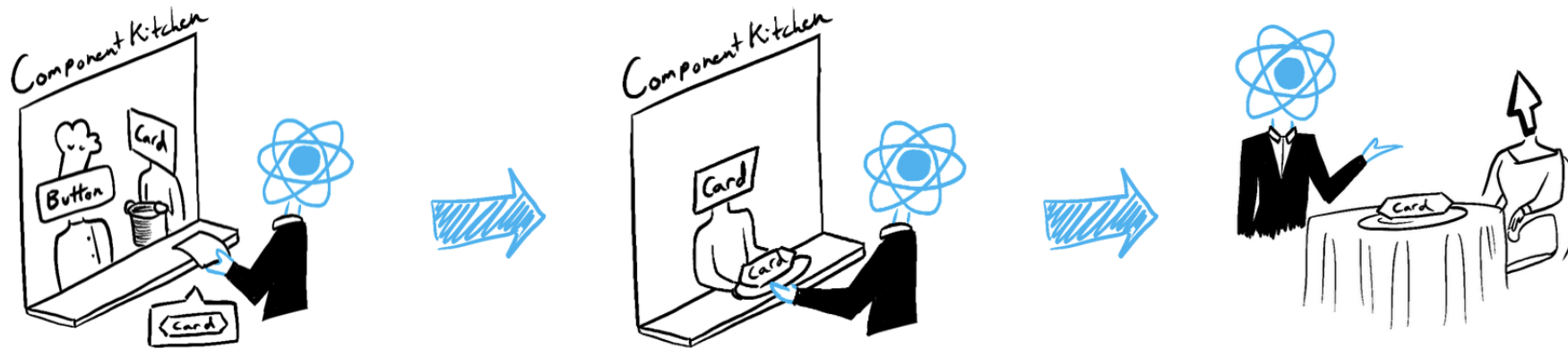
Every hook asks React to keep something for you

hook	what React keeps for you	when you'll meet it
<code>useState</code>	a value — and re-renders when it changes	now: every drill this week
<code>useRef</code>	a value that doesn't re-render; or a DOM node, e.g. to focus an input	when you need it
<code>useEffect</code>	code to run after render, to sync with something outside React	session 22: fetching from your API
<code>useContext</code>	a value read from far above, without passing props down every level	group projects
<code>useReducer</code>	state plus named transitions — drill 9's status, grown up	optional
<code>useMemo</code>	a cached computed value — still derived, never stored	rarely; measure first
<code>useQuantity()</code>	your own hook, built from the ones above	drill 14

A custom hook shares logic, not state. Every component that calls `useQuantity()` gets its own slots — so sharing a number is still drill 2's job: lift it.

Requesting and serving UI has three steps

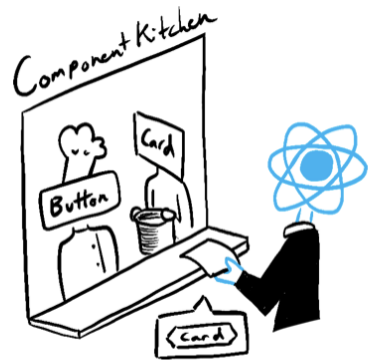
Trigger work. Call components to calculate the UI. Commit only the DOM changes needed.



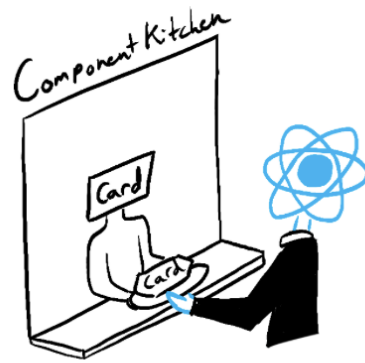
Step 1: Trigger a render

There are two reasons for a component to render:

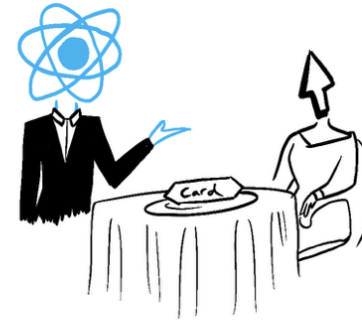
1. It's the component's **initial render**.
2. The component's (or one of its ancestors') **state has been updated**.



Trigger



Render

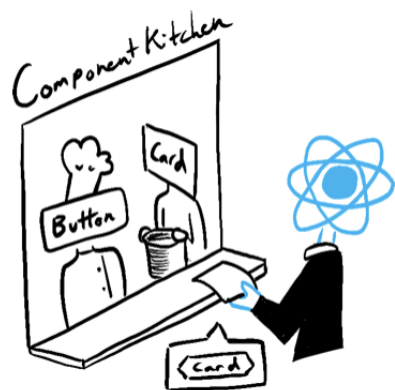


Commit

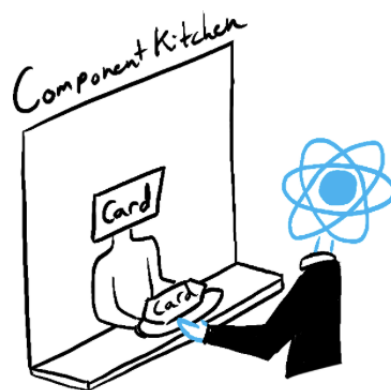
Step 2: React renders your components

After you trigger a render, React calls your components to figure out what to display on screen. **“Rendering” is React calling your components.**

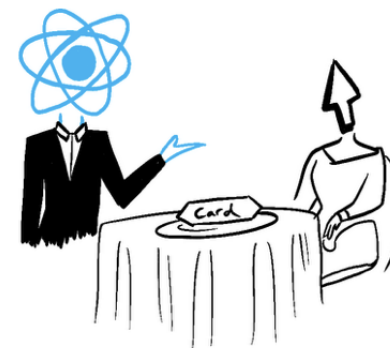
- On initial render, React will call the root component.
- For subsequent renders, React will call the function component whose state update triggered the render.



Trigger



Render



Commit

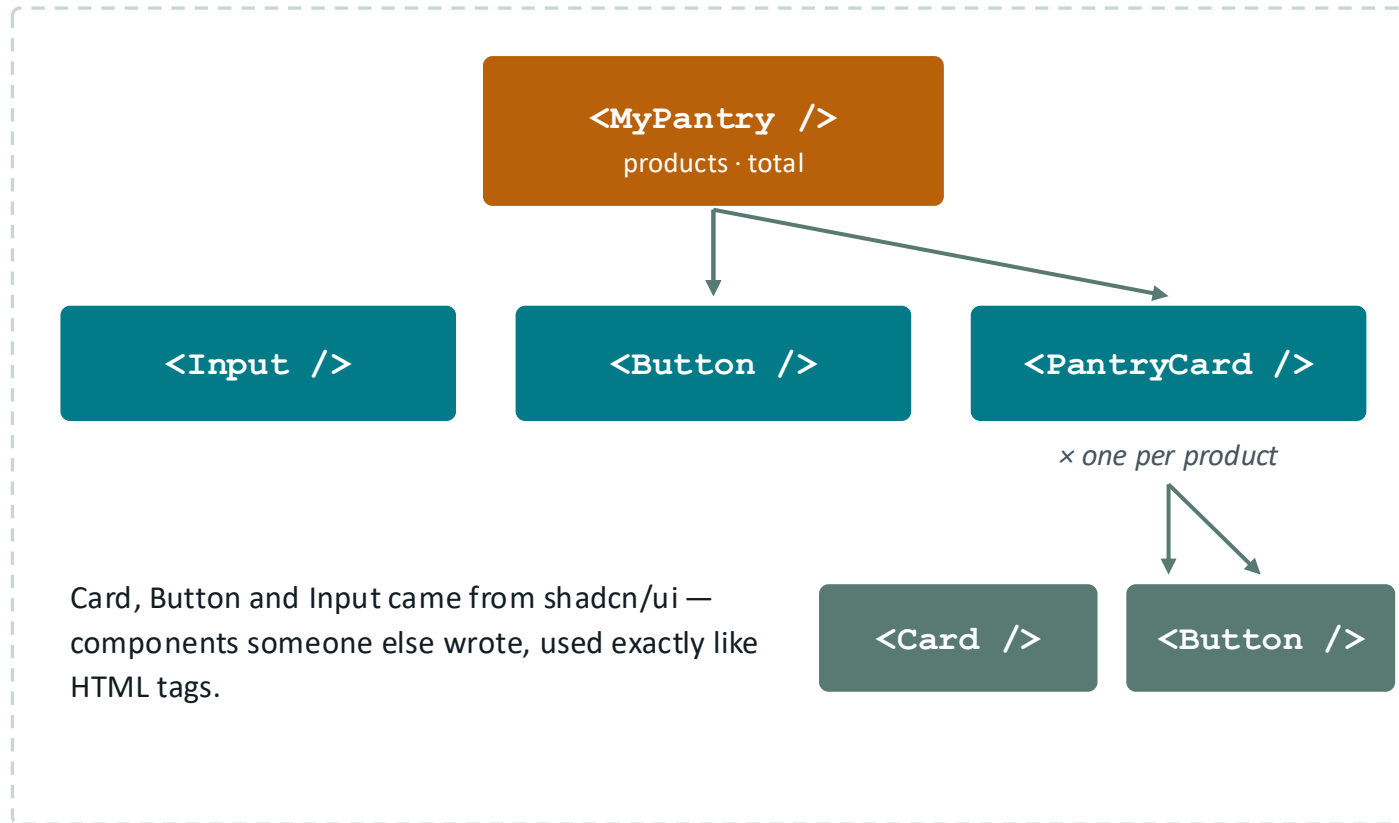
Step 3: React commits changes to the DOM

After rendering (calling) your components, React will modify the DOM.

- **For the initial render**, React will use the `appendChild()` DOM API to put all the DOM nodes it has created on screen.
- **For re-renders**, React will apply the minimal necessary operations (calculated while rendering!) to make the DOM match the latest rendering output.

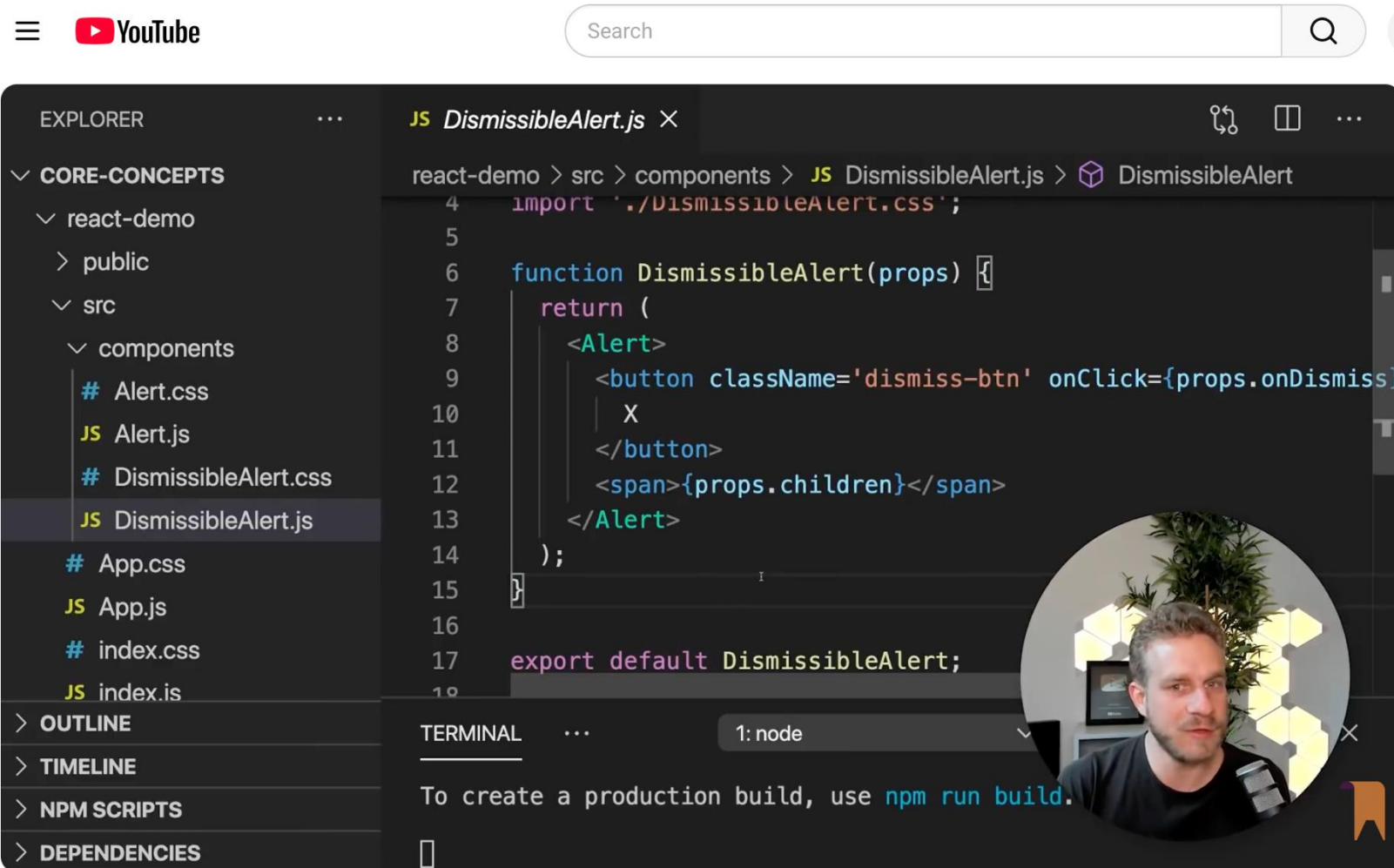
COMPOSITION

A page is a tree of components



- Data goes down as props: MyPantry hands each PantryCard its name and quantity.
- Changes go up as calls: a card's button calls the onAdd it was given, and MyPantry updates products.
- The state sits at the top because MyPantry is the only component that sees every product.
- **Component composition** is the fundamental pattern in React where you build complex user interfaces by combining smaller, isolated components rather than relying on class inheritance. React adheres to the philosophy of "**composition over inheritance**," meaning you should pass components as data or nest them together to share UI and logic.

This is the tree HW4 asks you to build (due Friday). Drill 11 was one branch of it.



<https://www.youtube.com/watch?v=of7isf3YBZ4>

ITWS 2110 · WEEK 6 · SESSIONS 10 AND 11

React Part II

Last week you made things that respond to a click. This week: deciding where that response should live, and then proving it stays correct without you watching the screen.

Tuesday is mostly lecture and live debugging. Friday is a lab, and it feeds Homework 5.

You can already build a counter. Now what?

- Components, props, lists, conditions, useState, controlled inputs — session 9 gave you a working vocabulary.
- It also gave you enough rope: two counters that should agree and don't, a value you could compute but stored instead, a component that only knows how to render one thing.
- None of those are new concepts today. They are the same ideas from last week, under load — more than one component, more than one piece of state, a real decision about where something lives.

Today is not new syntax. It is the first time “does this work?” stops being enough, and “is this the right place for it?” starts mattering.

Four Design Issues with React

1 **Keys**

identity across a re-render, not a label for humans (Drill 1)

2 **Lifting state up**

to the lowest component that contains everyone who reads it (Drills 2-3)

3 **Derived state**

compute it if you can; don't give it a second useState (Drill 4)

4 **Composition**

children when the shape of the content isn't yours to know (Drill 5)

None of this has a compiler error for getting it wrong. That is exactly why it's worth a whole session instead of a paragraph in last week's deck.

Drills 1-4 in [inclass/wk6/react](#) cover all four. Do them now if time allows, or before Friday if it doesn't.

Index keys confuse position with identity

AFTER A REORDER, THE KEY STAYS WITH THE SLOT

BEFORE

0	Rice	key="0"	note="white"
1	Beans	key="1"	note="black"

MOVE BEANS TO THE FRONT

0	Beans	key="0"	← reuses Rice's row
1	Rice	key="1"	← reuses Beans' row

- A key tells React which rendered item is the same item on the next render.
- An index identifies a position in the list — not the item occupying it.
- After a reorder, key “0” still belongs to position 0, even though a different item is there.
- React may reuse the wrong row, so an uncontrolled input can keep the previous item's value.

The label updates, but the input can keep the old row's value — identity and position have been confused.

Stable IDs keep state attached to the right item

USE THE ID ALREADY IN YOUR DATA

```
{items.map(item => (  
  <li key={item.id}>  
    <input defaultValue={item.note} />  
  </li>  
))}  
  
// Beans keeps key "beans-42" before and  
// after the move. React moves the right row.
```

- A stable key follows the data item, even when its position changes.
- React can match Rice with Rice and Beans with Beans across renders.
- The correct DOM node and component state move with each item.
- Use an index only when the list truly cannot reorder, insert, or delete items.

Rule of thumb: if the data has an id, use that id as the key.

Two things that must agree, and don't

`<Stepper />`
qty: 2

`<Stepper />`
qty: 0

Same product, shown twice. Each Stepper owns its own `useState`, so they can only agree by accident.

`<Drill2 />`
qty: 2

`<Stepper />`

`<Stepper />`

quantity moves to the closest shared parent. Both children read it as a prop and call one shared setter.

The rule: state moves up to the lowest component that contains everyone who needs to read it. Not higher, not lower.

Drill 2 starts broken like the left side. The refactor is the drill, not just reading the fixed version.

DERIVED VS. STORED STATE

If you can compute it, don't store it

STORED — TWO FACTS THAT CAN DISAGREE

```
const [items, setItems] = useState(['Apples']);
const [total, setTotal] = useState(1);

function removeLast() {
  setItems(list => list.slice(0, -1));
  // total is never told. It is now wrong.
}
```

DERIVED — ONE FACT, COMPUTED FRESH

```
const [items, setItems] =
  useState(['Apples']);
const total = items.length;

function removeLast() {
  setItems(list => list.slice(0, -1));
  // total is right, because it was never
  // a separate fact to begin with.
}
```

Most state bugs are not wrong logic. They are one true fact, copied into two variables, that someone forgot to update in one place.

The question to ask before every useState: could this value be computed from state or props I already have?

THE HOOK YOU'LL BE TEMPTED BY

useEffect is not a sync button

AN EFFECT THAT COPIES — STILL TWO FACTS

```
const [items, setItems] = useState(['Apples']);
const [total, setTotal] = useState(0);

useEffect(() => {
  setTotal(items.length);
}, [items]);

// Draws a stale total, then renders again.
// The copy is still there -- now a third
// piece of code exists to chase it.
```

DURING RENDER — NOTHING TO CHASE

```
const [items, setItems] = useState(['Apples']);
const total = items.length;
```

- This is the fix an AI assistant or a search result will hand you for drills 3, 12 and 13. It works, and it's the pattern the reading tells you to avoid.
- An effect runs after React has already drawn the screen. It is for talking to things outside React — a server, a timer, localStorage.

If you can compute it during render, you don't need an effect. Effects are for the outside world — and fetching from your own API is session 22.

react.dev/learn/you-might-not-need-an-effect — read the first section once; it's this slide, with more examples.

COMPOSITION

children instead of twenty props

CONFIGURATION — ONE PROP PER POSSIBLE FOOTER

```
function Card({ title, footerText, onFooterClick }) {  
  return (  
    <div>  
      <h3>{title}</h3>  
      <button onClick={onFooterClick}>  
        {footerText}  
      </button>  
    </div>  
  );  
}  
// Needs a SECOND button? secondFooterText,  
// onSecondFooterClick... and it never ends.
```

COMPOSITION — CARD DOESN'T ASK

```
function Card({ title, children }) {  
  return (  
    <div>  
      <h3>{title}</h3>  
      {children}  
    </div>  
  );  
}  
  
<Card title="Rice">  
  <button>Add one</button>  
  <button>Remove one</button>  
</Card>
```

Drill 4: Card can't take a second button no matter what prop you invent. Give it children instead.

FROM THE READINGS

Sharing State Between Components

Sometimes, you want the state of two components to always change together. To do it, remove state from both of them, move it to their closest common parent, and then pass it down to them via props. This is known as *lifting state up*, and it's one of the most common things you will do writing React code.

You will learn

- How to share state between components by lifting it up
- What are controlled and uncontrolled components

<https://react.dev/learn/sharing-state-between-components>

```
1 import { useState } from 'react';
2
3 function Panel({ title, children }) {
4   const [isActive, setIsActive] = useState(false);
5   return (
6     <section className="panel">
7       <h3>{title}</h3>
8       {isActive ? (
9         <p>{children}</p>
10      ) : (
11        <button onClick={() => setIsActive(true)}>
12          Show
13        </button>
14      )}
15     </section>
16   );
```

▼ Show more

Almaty, Kazakhstan

About

Show

Etymology

Show

children lets a component reveal any nested UI

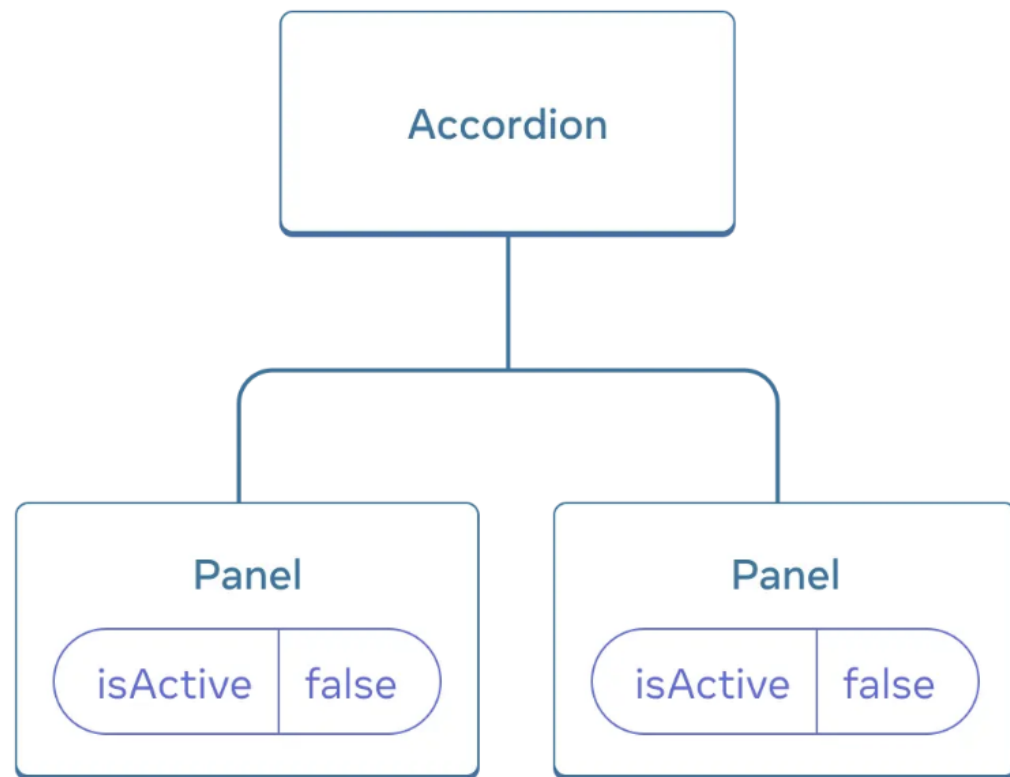
children — CONTENT PASSED BETWEEN THE TAGS

```
function Panel({ title, children }) {  
  const [open, setOpen] = useState(false);  
  return <section>  
    <h3>{title}</h3>  
    {open ? children : (  
      <button onClick={() => setOpen(true)}>  
        Show  
      </button>  
    )}  
  </section>;  
}
```

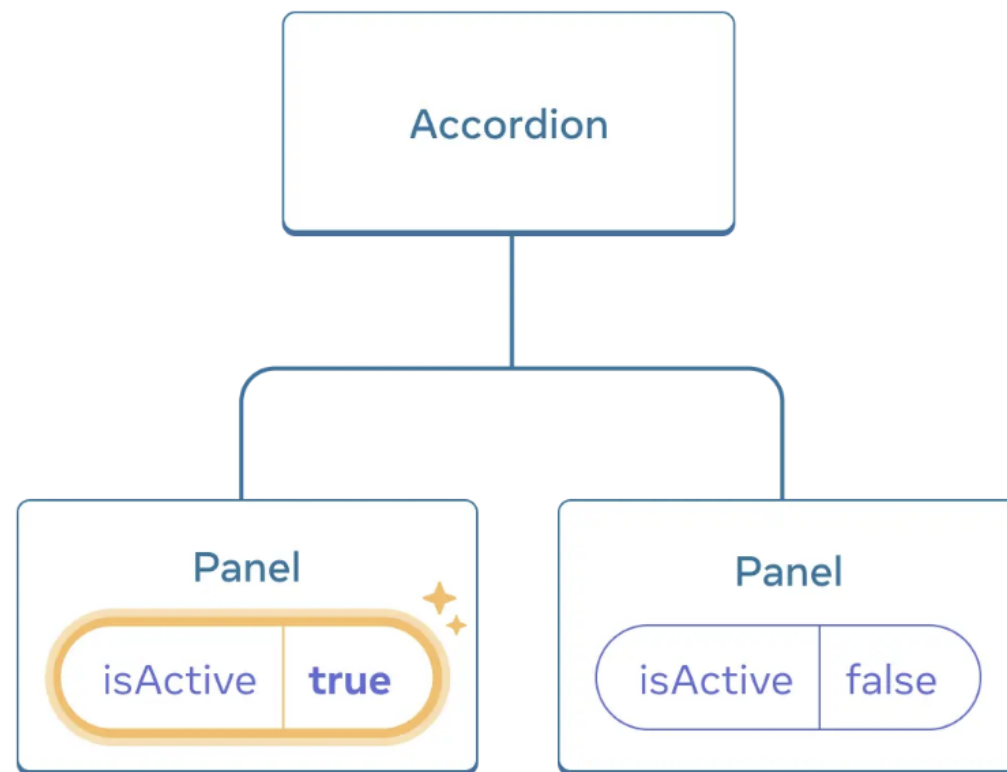
- React passes everything between <Panel> and </Panel> as the special children prop.
- Panel decides where to place that content — and here, when to reveal it.
- Each instance can receive different content while sharing the same heading, border, and Show behavior.
- This is composition: reusable structure outside, flexible content inside.

Use children when the wrapper owns the layout or behavior, but the parent should supply the content.

Notice how pressing one panel's button does not affect the other panel—they are independent.



Initially, each Panel's `isActive` state is `false`, so they both appear collapsed



Clicking either Panel's button will only update that Panel's `isActive` state alone

Choosing the State Structure

Structuring state well can make a difference between a component that is pleasant to modify and debug, and one that is a constant source of bugs. Here are some tips you should consider when structuring state.

You will learn

- When to use a single vs multiple state variables
- What to avoid when organizing state
- How to fix common issues with the state structure

<https://react.dev/learn/choosing-the-state-structure>

Principles for structuring state

When you write a component that holds some state, you'll have to make choices about how many state variables to use and what the shape of their data should be. While it's possible to write correct programs even with a suboptimal state structure, there are a few principles that can guide you to make better choices:

1. **Group related state.** If you always update two or more state variables at the same time, consider merging them into a single state variable.
2. **Avoid contradictions in state.** When the state is structured in a way that several pieces of state may contradict and “disagree” with each other, you leave room for mistakes. Try to avoid this.
3. **Avoid redundant state.** If you can calculate some information from the component's props or its existing state variables during rendering, you should not put that information into that component's state.
4. **Avoid duplication in state.** When the same data is duplicated between multiple state variables, or within nested objects, it is difficult to keep them in sync. Reduce duplication when you can.
5. **Avoid deeply nested state.** Deeply hierarchical state is not very convenient to update. When possible, prefer to structure state in a flat way.

Group related state

You might sometimes be unsure between using a single or multiple state variables.

Should you do this?

```
const [x, setX] = useState(0);  
const [y, setY] = useState(0);
```

Or this?

```
const [position, setPosition] = useState({ x: 0, y: 0 });
```

Instead of

App.js

```
1  import { useState } from 'react';
2
3  export default function FeedbackForm() {
4    const [text, setText] = useState('');
5    const [isSending, setIsSending] = useState(false);
6    const [isSent, setIsSent] = useState(false);
7
8    async function handleSubmit(e) {
9      e.preventDefault();
10     setIsSending(true);
11     await sendMessage(text);
12     setIsSending(false);
13     setIsSent(true);
14   }
15 }
```

Use

App.js

```
4    const [text, setText] = useState('');
5    const [status, setStatus] = useState('typing');
6
7    async function handleSubmit(e) {
8      e.preventDefault();
9      setStatus('sending');
10     await sendMessage(text);
11     setStatus('sent');
12   }
13
14   const isSending = status === 'sending';
15   const isSent = status === 'sent';
16 }
```

Preserving and Resetting State

State is isolated between components. React keeps track of which state belongs to which component based on their place in the UI tree. You can control when to preserve state and when to reset it between re-renders.

You will learn

- When React chooses to preserve or reset the state
- How to force React to reset component's state
- How keys and types affect whether the state is preserved

<https://react.dev/learn/preserving-and-resetting-state>

State is tied to a position in the render tree

React builds [render trees](#) for the component structure in your UI.

When you give a component state, you might think the state “lives” inside the component. But the state is actually held inside React. React associates each piece of state it’s holding with the correct component by where that component sits in the render tree.

Here, there is only one `<Counter />` JSX tag, but it’s rendered at two different positions:

One bug per section of the reading

#	reading section	the bug you'll see	the fix
9	Choosing the State Structure — avoid contradictions	an order is “Sending...” and “Sent!” at once	<code>one status</code>
10	Choosing the State Structure — avoid duplication	rename the product you picked; “You picked” goes stale	<code>store the id</code>
11	Sharing State Between Components — the accordion	two panels that should take turns both open	<code>lift activeIndex</code>
12	Preserving and Resetting State — reset with a key	switch products; the note box keeps the old note	<code>key={selected.id}</code>
13	Choosing the State Structure — don't mirror props	the price changes; the tag doesn't	<code>no state at all</code>

Every one of these works until the data changes. None of them has an error message — which is why the reading spends a whole chapter on them.

One status, not two booleans

THE REQUEST

Click Send order, then Mark delivered: the page claims it's sending and sent. Replace `isSending` and `isSent` with one status.

WHAT IT DEMONSTRATES

Two booleans make four combinations, and one of them — both true — is impossible. One value with three states can't contradict itself.

READ FIRST

Choosing the State Structure — avoid contradictions in state

STARTER — THE BUG

```
const [isSending, setIsSending] = useState(false);
const [isSent, setIsSent] = useState(false);
...
{isSending && <p>Sending...</p>}
{isSent && <p>Sent!</p>}
```

THE FIX

```
const [status, setStatus] = useState('typing');
// 'typing' -> 'sending' -> 'sent'
{status === 'sending' && <p>Sending...</p>}
{status === 'sent' && <p>Sent!</p>}
```

Done when after both clicks, the page says only “Sent!”.

Explain: adding `setIsSending(false)` to the second button also passes. Why is one status still the better fix?

DRILL 10 · FROM THE READINGS

Store the id, not a copy

THE REQUEST

Choose Rice, then rename it in its box. The list changes; “You picked” doesn't. Keep `selectedId` in state and look the product up.

WHAT IT DEMONSTRATES

`selected` was a second copy of an object that already lives in `items`. Store the smallest thing that identifies it — the `id` — and derive the rest every render.

READ FIRST

Choosing the State Structure — avoid duplication in state

STARTER — THE BUG

```
const [selected, setSelected] = useState(pantry[0]);  
...  
<button onClick={() => setSelected(item)}>Choose  
<p>You picked: {selected.name}</p>
```

THE FIX

```
const [selectedId, setSelectedId] = useState('a');  
const selected = items.find(i => i.id === selectedId);  
...  
<button onClick={() => setSelectedId(item.id)}>
```

Done when choose Rice, rename it Brown rice — “You picked: Brown rice” follows.

Explain: this is drill 3 from another direction. What's the one fact, and where was the second copy?

Only one panel open

THE REQUEST

Show Storage, then Shelf life: both stay open. Lift the state: Drill11 keeps `activeIndex`, and each Panel gets `isActive` and `onShow`.

WHAT IT DEMONSTRATES

Two panels that must take turns need one owner. It's drill 2 again: the state moves to the closest parent both panels share.

READ FIRST

Sharing State Between Components — lifting state up by example

STARTER — THE BUG

```
function Panel({ title, children }) {  
  const [isActive, setIsActive] = useState(false);  
  ...  
}  
  
<Panel title="Storage">...</Panel>
```

THE FIX

```
function Panel({ title, isActive, onShow, children })  
const [activeIndex, setActiveIndex] = useState(null);  
<Panel isActive={activeIndex === 0}  
  onShow={() => setActiveIndex(0)} ...>
```

Done when show Storage, then Shelf life — only Shelf life is open.

Explain: the reading calls the new Panel “controlled”. Controlled by what?

Reset with a key

THE REQUEST

Click Rice: the label says Rice, but the box still holds the Apples note. Give `<NoteEditor>` a key.

WHAT IT DEMONSTRATES

Same component, same place in the tree: React keeps its state, and `useState(product.note)` only reads its argument the first time. A new key tells React it's a different editor.

READ FIRST

Preserving and Resetting State — resetting a form with a key

STARTER — THE BUG

```
function NoteEditor({ product }) {  
  const [text, setText] = useState(product.note);  
  // reads product.note on the FIRST render only  
}  
  
<NoteEditor product={selected} />
```

THE FIX

```
<NoteEditor key={selected.id} product={selected} />  
// a different product -> a different editor,  
// with fresh state
```

Done when after clicking Rice, the box says “buy the 5lb bag”.

Explain: drill 1 used a key to keep state with the right item. What is this key doing instead?

Read the prop, don't copy it

THE REQUEST

Click Start the sale: the price drops to \$3, but the tag still says \$4. PriceTag only shows the price — it needs no state.

WHAT IT DEMONSTRATES

`useState(price)` copies the prop once and ignores it from then on. A component that only displays a value should read the prop every render.

READ FIRST

Choosing the State Structure — don't mirror props in state

STARTER — THE BUG

```
function PriceTag({ price }) {  
  const [shown] = useState(price); // copied once  
  return <p>${shown}</p>;  
}
```

THE FIX

```
function PriceTag({ price }) {  
  return <p>${price}</p>;  
}
```

Done when click Start the sale — the tag says \$3.

Explain: 12 and 13 are the same trap. Why is the fix a key in one and “no state” in the other?

Same trap, two different fixes

```
const [x, setX] = useState(prop);  
// reads prop on the first render -- never again
```

12 · the user edits it

- The note is a draft — the component needs its own editable copy.
- So keep the state. Reset it when the product changes: `key={selected.id}`.

13 · nobody edits it

- The tag only shows the price. There's nothing to keep.
- So delete the state and read `{price}` every render.

Ask one question: does this component own an editable copy, or does it just show what it's given? Neither answer is `useEffect`.

A student who can explain why these two fixes differ has understood this week's reading.

A custom hook shares logic, not state

THE REQUEST

Click + next to Apples twice: the shelf says 2, the cart says 0. Leave `useQuantity` alone — call it in `Drill14`, once per product, and pass the results down.

WHAT IT DEMONSTRATES

Every call to a hook gets its own `useState`. Four calls to `useQuantity()` are four separate numbers. Sharing a number is still drill 2's job: lift it.

READ FIRST

Reusing Logic with Custom Hooks — share stateful logic, not state

STARTER — THE BUG

```
function ShelfLabel({ name }) {  
  const { quantity, add, remove } = useQuantity();  
}  
function CartLine({ name }) {  
  const { quantity } = useQuantity(); // separate!  
}
```

THE FIX

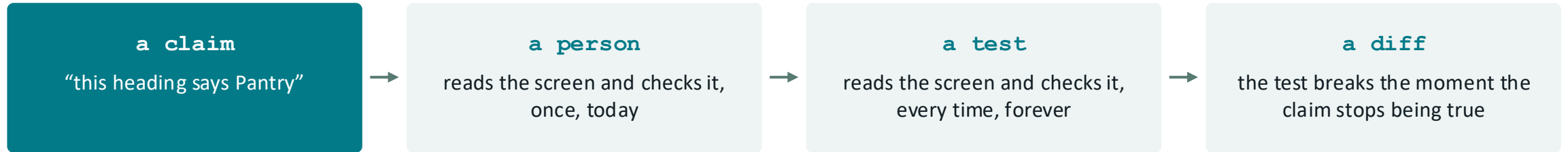
```
const apples = useQuantity(); // in Drill14  
const rice = useQuantity();  
<ShelfLabel quantity={apples.quantity}  
  onAdd={apples.add} ... />  
<CartLine quantity={apples.quantity} ... />
```

Done when + on Apples twice and Rice once — the cart says Apples 2, Rice 1.

Explain: you call `useQuantity()` twice in `Drill14` and want the numbers separate. What decides whether two things share one call?

Component Tests

The same argument from session 4, made visible



Last week's drill 1 and this week's drill 5 make the same kind of claim about a heading. One is checked by looking. The other is checked by a machine, on every save, forever.

- Nothing about the components changes today. `pantry.jsx` already works — you write the description, not the fix.
- This is the same behavior-not-implementation argument as session 4's TDD unit: a test names a claim a user could verify by looking, and checks nothing else.

This is the mechanism Homework 5 grades with — a runner, and a human, on a fifth test you write yourself.

Query the page the way a person reads it

```
render(<ProductHeading />);

// A person doesn't see class names. Neither
// should this test.
const heading = screen.getByRole('heading', {
  level: 2,
});
expect(heading).toHaveTextContent('Pantry');
```

- `render()` puts the component in a real, throwaway DOM. screen queries that DOM.
- `getByRole` finds elements the way a screen reader or a mouse-and-eyes person would — by what they are, not by a class you typed.
- Query priority, roughly: role, then label text, then visible text. Class names and test-only attributes are the last resort, not the first.
- A test that queries by class breaks when you restyle. A test that queries by role breaks only when the page stops making sense to a person.

act, then assert — in that order, awaited

```
const user = userEvent.setup();
render(<AddButton />);

await user.click(screen.getByRole('button'));
await user.click(screen.getByRole('button'));

expect(screen.getByRole('button'))
  .toHaveTextContent('Added 2');
```

- `user.click` simulates a real click, then waits for React to finish re-rendering — that wait is why it returns a promise.
- Skip the `await` and the assertion runs against the DOM from before the click landed. The test sees Added 0 and the failure looks like a broken component.
- This is the single most common Homework 5 failure. It looks like a bug in the code under test. It never is.
- One `await` per action, always — two clicks need two `awaited` calls, not one click `awaited` twice.

If a test's failure message looks impossible — the screen shows the right thing but the assertion disagrees — check for a missing `await` before anything else.

getByLabelText and user.type

```
render(<ProductNameField />);  
const field = screen.getByLabelText('Product name');  
  
await user.clear(field);  
await user.type(field, 'Oats');  
  
expect(field).toHaveValue('Oats');
```

- `getByLabelText` finds the input through its `<label>`, the same `htmlFor/id` pair from drill 10 — accessibility and testability, same property.
- `user.type` appends to whatever is already there. Clear first, or the assertion sees Apples Oats.
- Change the label text in `pantry.jsx` and this test breaks. That coupling is the point: the label is the contract with the person using the page.

A label is not decoration. It is how a screen reader announces the field, how a click on the text focuses it, and now, how the test finds it.

React with tests you write yourself

- Homework 4 is due today — drills 1-11 and your pantry, with a list, add, remove, and a computed total.
- Homework 5 asks you to write component tests of your own, graded by a runner and by a human reading them.
- The four testing drills from today are the whole shape of what it asks: render, query by role or label, act and await, assert.
- Nothing new to build — the components already exist. The skill being graded is stating a true, checkable claim and proving it.